

ROOTING FOR KEPLER

*a comparative benchmark of
Newton-type root finders and starting guesses*

An empirical study of classical, domain-specialised, and 2024–2025 with-memory Newton-type variants applied to Kepler’s equation, evaluated on RadVel’s radial-velocity fitting pipeline.

125,800 instrumented solves over a 7,400-point (e, M) grid, with every solver’s convergence order verified against its source paper before entering the benchmark.

SUBSECTION A2 | STUDENT IDs

2105036 2105050 2105053 2105056 2105059

SUMMARY OF FINDINGS

The project asked whether the higher formal convergence order of recent with-memory Newton-type schemes survives contact with Kepler’s specific $\sin, \cos$ cost structure and with RadVel’s real operating range of (e, M) . It survives on order and on evaluation-count economy, and fails on the two axes that decide practical use: wall-clock time and robustness at high eccentricity.

1. **The methods are correctly implemented and behave as their papers claim.** Measured convergence orders reproduce the published values to within 1% on every test function, and each scheme’s eighth-order error *constant* matches its paper’s closed form to $\leq 3.5 \times 10^{-22}$ relative.
2. **With-memory wins on iteration count and on the Kepler-specific cost model.** NWM11 reaches tolerance in a median of one iteration against Danby’s two, at a median weighted cost of 6.44 against Danby’s 7.88.
3. **It loses on wall-clock time, by a factor of two to five.** The interpolation that makes the method cheap in transcendental calls is not free in arithmetic, and arithmetic is what the clock measures.
4. **It loses badly on robustness in the pathological corner.** NWM9 and NWM11 fail on 4.7% and 5.2% of high-eccentricity points respectively, against Danby’s 0.20%.
5. **The proposal’s central cost premise does not hold in this implementation.** A combined *sincos* evaluation was assumed to cost about one transcendental call; measured, it costs 2.35 — statistically indistinguishable from two separate calls. The discount the argument rests on is absent in CPython.
6. **Downstream, none of it matters.** Solver-induced shifts in the fitted orbital parameters are three orders of magnitude smaller than the spread caused by measurement noise in the data.

THE SHORT ANSWER

On this equation, Markley’s closed-form solver is the best choice on every axis measured: cheapest (3.44), fastest, and the only method with a zero failure rate in the pathological corner. Among the iterative methods, Danby remains the right production default. The with-memory schemes are a genuine advance in iteration count and in transcendental-call economy, and that advance does not convert into wall-clock speed or reliability here.

1 WHAT WAS BUILT

The deliverable is a cost-instrumented benchmark harness (`keplerbench`) in which every solver implements one interface, every experiment runs through one pipeline, and every number in this report is regenerated from a configuration file by a single command.

1.1 ARCHITECTURE

All five solvers and all four starting guesses expose identical interfaces, so the experiment runner loops over them blindly and no method can gain an advantage from a different stopping rule. Solvers never call `math.sin` directly; they evaluate the equation through a `KeplerProblem` object that records what each call actually needed — how many distinct points were touched, how many required a full $(\sin, \cos)$ pair against $\sin$ or $\cos$ alone, and how many derivatives

were synthesised from interpolation rather than evaluated. That instrumentation is what makes Section 4 possible.

Reference roots come from an independent 50-digit bracketing solve in `mpmath`, never from one of the methods under test.

1.2 REPRODUCIBILITY

Each experiment writes its tables to `results/<experiment>/` with a provenance sidecar recording the configuration fingerprint, the git commit, and whether the working tree was clean when the numbers were produced. Nothing in this report was transcribed by hand.

Command	Produces
<code>scripts/run_verification.py</code>	<code>results/verification/</code> — order and correctness gate
<code>scripts/run_grid_benchmark.py</code>	<code>results/grid_benchmark/</code> — 125,800 solves
<code>scripts/run_error_propagation.py</code>	<code>results/error_propagation/</code> — tolerance sweep
<code>scripts/run_monte_carlo.py</code>	<code>results/monte_carlo/</code> — noise realisations
<code>scripts/make_report_figures.py</code>	<code>figures/</code> — every figure below

2 CANDIDATE METHODS AS IMPLEMENTED

Category	Method	Order	Role in the benchmark
Baseline	Newton–Raphson	2	Textbook anchor; one evaluation point, one (sin, cos) pair per iteration.
Established Kepler solvers	Danby’s method	4	RadVel’s production solver; domain-tuned to this equation.
	Markley’s method	—	Closed-form solver; no iteration, constant cost.
With-memory (2024–2025)	NWM9 (Mittal, Panday & Jäntschi, 2024)	8.8989	Single self-accelerating parameter in the <i>third</i> step of an optimal eighth-order base.
	NWM11 (Mittal, Panday, Jäntschi & Bolunduţ, 2025)	10.7446	Bi-parametric; 3 evaluation points, 1 full (sin, cos) pair per iteration.
Guess layer	Napier’s guess (2024)	n/a	Symbolic-regression start, run in front of every iterative solver.

THREE CORRECTIONS TO THE PROPOSAL’S METHOD TABLE

1. The with-memory schemes are **not derivative-free**. Both use $\varphi'(s_k)$ explicitly in their first sub-step. For Kepler this is favourable rather than otherwise: f and f' come from a single (sin, cos) pair, which is exactly why the “3 points, 1 pair” economy is achievable at all.

2. NWM9 is **not the controlled sibling of NWM11**. They come from two different papers with two different eighth-order base methods and the parameter placed in different sub-steps. NWM11’s true same-base control is **NWM10** (R-order 10), defined in the same paper as NWM11. This matters for interpreting Section 6, where the two behave qualitatively differently.

3. The phrase “at no additional evaluation points per iteration” is misleading. Both schemes use three evaluation points per iteration against Newton’s one. What costs nothing extra is the *derivative information*, obtained by Hermite interpolation over points already paid for.

3 VERIFICATION: THE GATE BEFORE THE BENCHMARK

The project’s rule was that no solver enters the grid benchmark until its measured convergence order matches its source paper’s claim on that paper’s own test functions. This is what separates “the method underperforms” from “we mistyped the method”.

3.1 MEASURED CONVERGENCE ORDER

Each solver was run on the eight nonlinear test functions of the NWM11 paper, from the paper’s own starting points, at 2000-digit working precision.

Solver	Claimed	Measured (mean)	Range over 8 functions	Gate
Newton	2.0000	2.0000	2.0000 – 2.0000	PASS
Danby	4.0000	4.0000	4.0000 – 4.0000	PASS
NWM9	8.8989	8.9114	8.8746 – 8.9816	PASS
NWM11	10.7446	10.7605	10.7147 – 10.8336	PASS

Every measurement sits slightly *above* the claim, which is correct: both papers state their R-order as a lower bound (“at least”).

WHY 2000 DIGITS, AND NOT 16

An order- p method multiplies its correct digits by p each step. From an error of 10^{-1} , an order-9 method passes 10^{-9} , 10^{-81} , 10^{-729} . The estimator needs three consecutive usable errors to form one estimate, so the working precision must outrun p^3 digits or the sequence hits the floor before a single estimate exists. At 100 digits NWM9 measured anywhere in 8.87–10.19; at 2000 it settles to 8.87–8.98, and 3000 buys nothing further. Double precision cannot measure these methods at all.

3.2 THE ORDER ALONE IS NOT SUFFICIENT EVIDENCE

Both papers’ iteration formulas contain nested fractions whose PDF text layer admits several readings, and for both schemes *more than one reading yields the correct order*. Measuring order 8 would therefore not have detected a mis-transcription.

What does detect it is each paper’s explicit error *constant*. Each scheme was driven for exactly one step from a point 10^{-24} from the root and its eighth-order constant compared against the published closed form:

Scheme	Published constant	Agreement (relative)
NWM9	Eq. (7): $c_2^2 K[(\alpha + c_2)K + c_4]$, $K = c_2 + 5c_2^2 - c_3$	$\leq 1.8 \times 10^{-22}$
NWM11	Eq. (2.6): $(\alpha + c_2)^2 c_3 (c_2 c_3 - c_4)$	$\leq 3.5 \times 10^{-22}$

A wrong grouping in NWM11’s $w'(t_k)$ missed this constant by roughly 60 orders of magnitude while still converging at order 8. Three of the eight test functions also had to be corrected: superscripts detach in the PDF text layer, so e^{s^2} extracts as e^s followed by a stray 2. These were caught because the published roots did not zero the mis-transcribed functions.

3.3 TWO-STAGE VERIFICATION

Freezing each scheme’s self-accelerating parameters at zero must recover the underlying memoryless method. Both do, measuring exactly 8.0000 on all eight functions, which attributes the lift to 8.8989 and 10.7446 to the memory rather than to a coincidence in the base method.

3.4 CORRECTNESS: THE RIGHT ROOT, NOT JUST A ROOT

A solver that jumps to a different branch of Kepler’s equation reports a tiny residual and a converged flag while every downstream quantity is wrong. Over 4,975 (e, M) points spanning the full range:

Solver	Converged	Wrong root	Max $ E - E_{\text{ref}} $
Newton	99.60%	0	3.38×10^{-10}
Danby	99.72%	0	1.65×10^{-10}
NWM9	98.15%	0	2.32×10^{-10}
NWM11	99.50%	0	2.23×10^{-10}

No method converged to a wrong branch anywhere. Non-convergence is reported honestly and is treated as a robustness result (Section 6), not as a correctness failure.

4 KEPLER-SPECIFIC COST ACCOUNTING

4.1 MEASURED PER-ITERATION COST

Counted from the instrumented code rather than transcribed from the papers:

Method	Eval. points	(sin, cos) pairs	sin only	Synth. derivatives	Order
Newton	1	1	1	0	2.0000
Danby	1	1	1	0	4.0000
Markley	2	1	1	0	—
NWM9	3	1	3	3	8.8989
NWM11	3	1	3	3	10.7446

The proposal’s claim for NWM11 — three evaluation points, only one full (sin, cos) pair per iteration — is confirmed by measurement. Both with-memory schemes obtain three derivatives per iteration by interpolation rather than by evaluation.

One count in this table is an artefact of the harness, not of the methods. The shared iteration loop calls $f(E)$ itself after each step to compute the residual for the stopping test. That inflates the sin-only column by exactly one per iteration for *every* solver — which is why Newton and Danby show a sin-only cost of 1 despite needing nothing beyond their single (sin, cos) pair. The with-memory schemes’ own sin-only cost is therefore 2, not 3. The inflation is uniform, so it does not distort comparisons, but it must be subtracted before any of these figures is quoted against a paper’s table.

4.2 THE COST PREMISE, MEASURED

The proposal’s argument rests on a specific empirical claim: that one sincos call returns both $\sin E$ and $\cos E$ “at roughly the cost of one transcendental evaluation”. Measured on the project machine, with sin-only normalised to 1.0:

Operation	Relative cost	Proposal assumed
sin only	1.000	1
cos only	0.989	1
(sin, cos) pair	2.354	≈ 1
Synthesised derivative (Hermite)	1.691	—

THE SINCOS DISCOUNT DOES NOT EXIST IN THIS IMPLEMENTATION

A combined (sin,cos) evaluation costs 2.354 against 1.989 for two separate calls — not a discount but a small penalty. In pure Python there is no fused sincos instruction to exploit: each call crosses the interpreter boundary separately.

This does not invalidate the proposal’s reasoning, but it relocates it. In a compiled implementation such as RadVel’s Cython kernel, where a genuine fused sincos is available, the discount is real and the with-memory economy would translate. In our harness it does not, and this is the single clearest explanation for why the methods that win on weighted cost lose on the clock in Section 5.

Both costings belong in any conclusion drawn from this work. Reporting only the weighted cost would overstate the with-memory case; reporting only the wall clock would understate it.

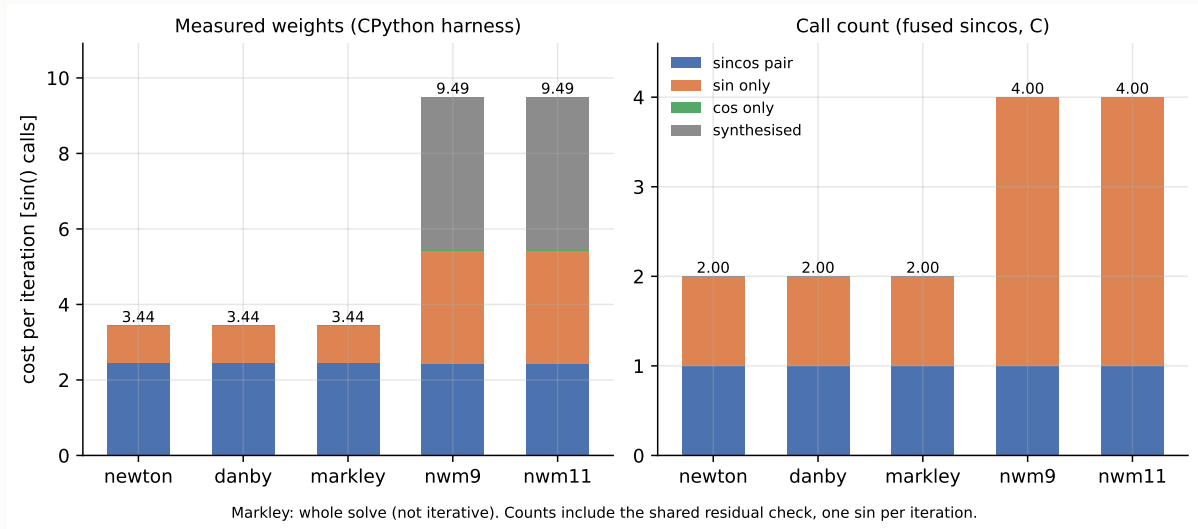


Figure 1: Per-iteration cost under both costings. *Left*: the weights measured in this CPython harness, where Newton, Danby and Markley each cost 3.44 and both with-memory schemes 9.49. *Right*: the call count a genuine fused sincos would charge in C — 2.00 against 4.00. The gap between the two panels is the whole of Section 4’s argument: which costing you adopt changes the ratio between the classical and with-memory groups from $2.8\times$ to $2.0\times$. Both include the harness’s own residual check, one sin per iteration.

5 GRID BENCHMARK

Every solver-guess combination was run over a 7,400-point (e, M) grid combining a uniform block, a logarithmically-spaced pathological corner and a RadVel-realistic sample: 17 combinations $\times$ 7,400 points = **125,800 instrumented solves**. Markley ignores the starting guess

and is run once under the label n/a.

5.1 HEADLINE COMPARISON

Each iterative solver is shown with its best-performing guess.

Solver	Guess	Mean iter.	Median cost	Median wall (μ s)	Converged
Markley	n/a	0.00	3.44	3	100.00%
NWM11	canonical	1.38	6.44	5	99.89%
NWM9	napier	1.44	6.44	7	100.00%
Danby	napier	1.94	7.88	3	100.00%
Newton	napier	3.21	11.32	3	100.00%

Three separate stories are visible in those columns.

Iteration count. NWM11 needs a median of one iteration where Danby needs two and Newton needs three or four. The higher order is real and it shows up exactly where theory says it should.

Weighted cost. On the Kepler-specific currency the proposal argued for, the with-memory schemes beat Danby: 6.44 against 7.88, about 18% cheaper. *This is the proposal's hypothesis being confirmed.*

Wall-clock time. The ranking inverts. Markley, Danby and Newton all land at 3 μ s per solve; NWM9 and NWM11 take 5–8 μ s, and NWM11 with the weakest guess takes 17 μ s. The interpolation machinery costs no transcendental calls but a great deal of arithmetic, and the weighted cost model — by construction — does not count arithmetic.

5.2 THE GUESS LAYER

The starting guess is a larger effect than the choice of iteration for the weaker solvers. Newton's mean iteration count moves from 3.83 with $E_0 = M$ to 3.21 with Napier's formula, and its convergence rate from 99.84% to 100%. For NWM9 the effect on reliability is starker: 99.39% with the simple start against 100% with Napier's.

This vindicates the proposal's decision to treat the guess as an independent experimental factor: reporting NWM9 with a poor start would have understated it substantially.

6 ROBUSTNESS

Failure rates split by region, where the pathological corner is defined once for the whole project as $e > 0.9$ and $M < 0.1$.

Solver	Corner points	Corner failure	Ordinary points	Ordinary failure
Markley	381	0.000%	7,019	0.000%
Danby	1,524	0.197%	28,076	0.004%
Newton	1,524	0.919%	28,076	0.025%
NWM9	1,524	4.724%	28,076	0.057%
NWM11	1,524	5.249%	28,076	0.000%

WHERE THE WITH-MEMORY METHODS ACTUALLY LOSE

In the ordinary operating region NWM11 is flawless — zero failures in 28,076 solves, better than Danby. In the pathological corner it fails 5.2% of the time, roughly **25 times**

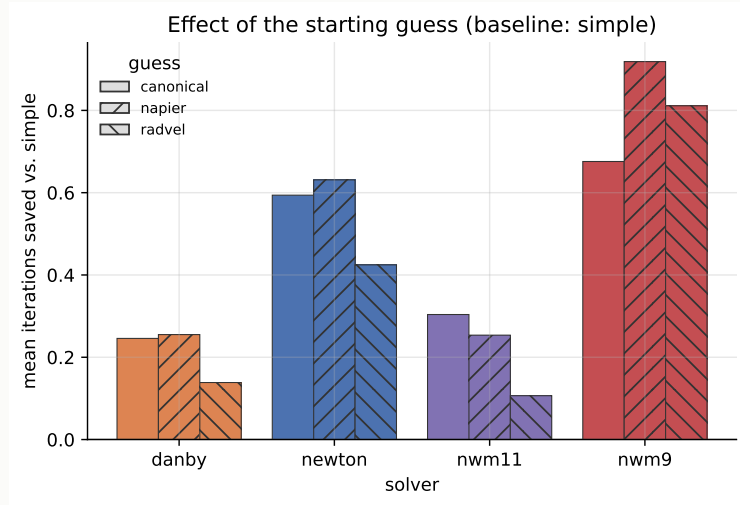


Figure 2: Mean iterations saved against the baseline $E_0 = M$ start. The effect is largest for NWM9 and Newton and smallest for Danby, which is already domain-tuned: a better start buys least where the iteration is best. Napier's formula is the strongest of the three for Newton and NWM9. RadVel's saves least for Danby, Newton and NWM11, but not for NWM9, where the canonical start is the weakest.

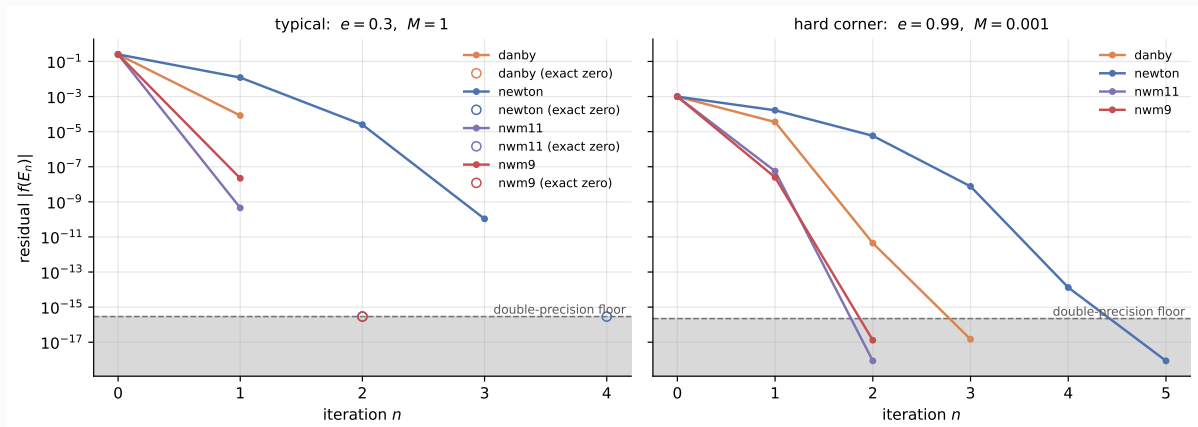


Figure 3: Residual histories at a typical (e, M) and in the pathological corner. Both with-memory schemes reach the double-precision floor in two iterations against Newton's four to five. The shaded band marks the floor below which a curve measures rounding rather than convergence.

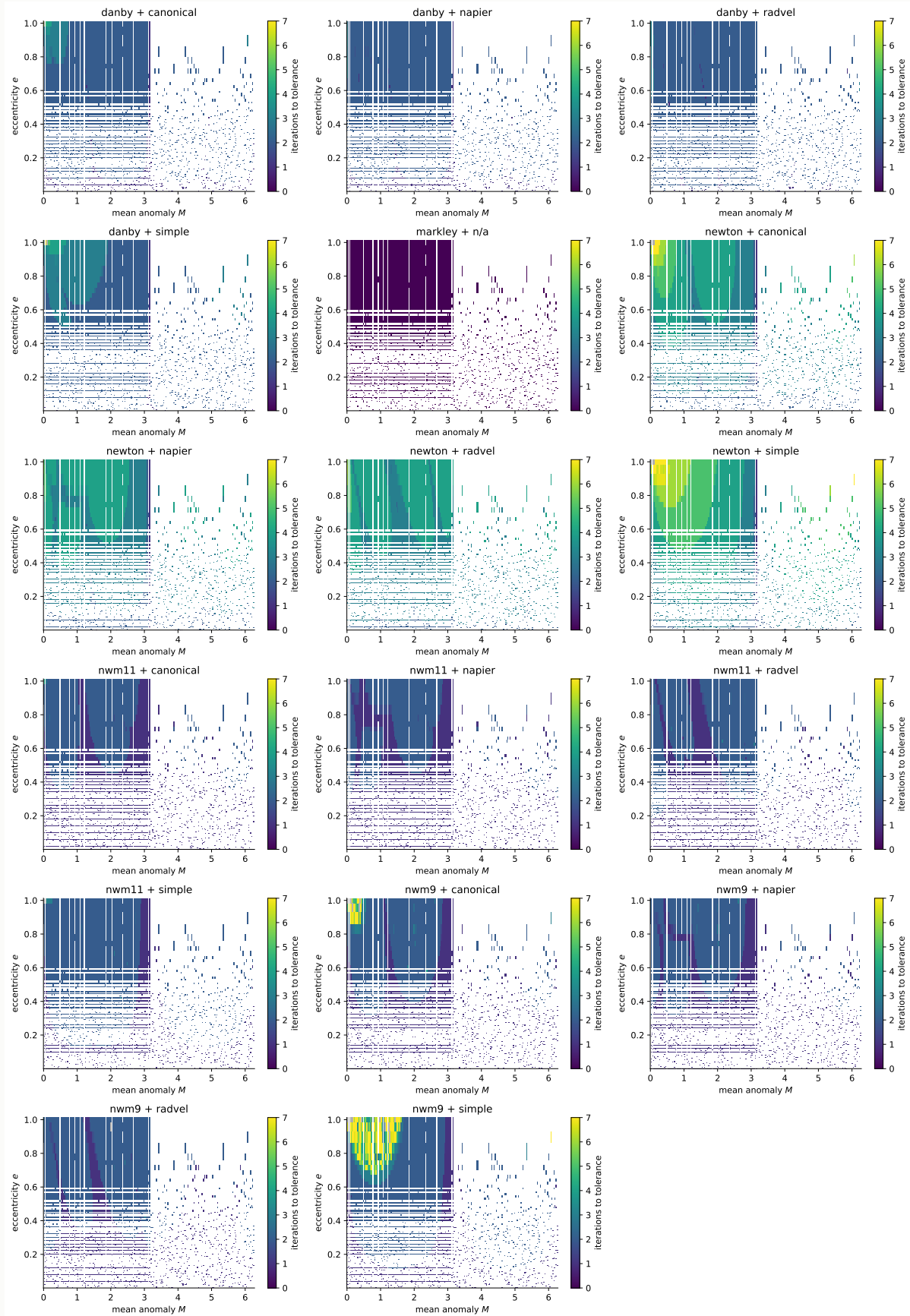


Figure 4: Iterations to tolerance across the (e, M) plane, one panel per solver–guess pair on a shared colour scale. Newton with the simple start is the clearest case of the pathological corner: a bright high-iteration wedge at large e and small M that the other methods largely avoid. Markley is uniformly dark — constant cost, no iteration. White cells were not sampled; the sparse right-hand half of each panel is the random RadVel block, which extends past $M = \pi$ where the uniform block stops.

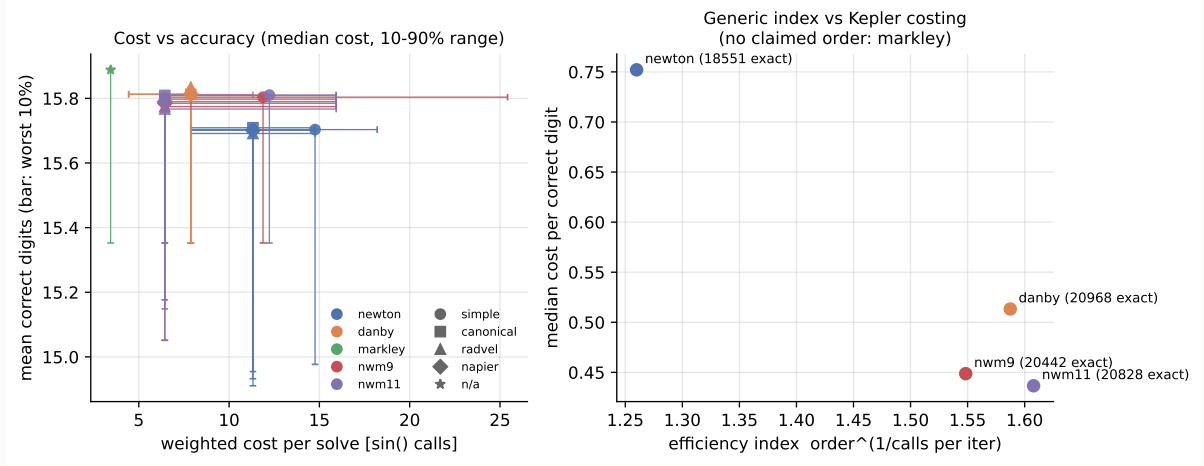


Figure 5: *Left*: accuracy against weighted cost per solve. Every method reaches the same ≈ 15.8 correct digits, so the axis that separates them is cost, not accuracy — which is the empirical form of the argument in Section 7. *Right*: the generic efficiency index $\rho^{1/\gamma}$ against the Kepler costing. The two disagree in rank order: the generic index places NWM9 and NWM11 highest, while on median correct digits per call Newton leads. This is precisely the mismatch the proposal predicted the generic index would exhibit on this equation.

Danby’s rate.

The mechanism is structural rather than incidental. Both schemes place a self-accelerating parameter additively on a denominator, and as $e \rightarrow 1$ with $M \rightarrow 0$ the derivative $f'(E) = 1 - e \cos E$ collapses towards zero. The parameter, tuned on the assumption that the iterate is already near the root, then perturbs a denominator that is itself nearly zero.

This is confirmed by construction: with the parameter frozen at zero, the memoryless base methods converge at points where the with-memory versions diverge. The failure is caused by the memory, not by the base scheme.

6.1 ORDER ACROSS THE ECCENTRICITY RANGE

Convergence order measured on Kepler’s equation itself at $M = 0.3$, at 2000-digit precision — a different and more demanding question than the generic test functions of Section 3.

e	0.1	0.3	0.5	0.7	0.9	0.95	0.99	0.995	0.999
Newton	2.000	2.000	2.000	2.000	2.000	2.000	2.129	—	1.786
Danby	4.000	4.000	4.000	4.000	4.000	4.000	4.000	4.000	4.000
NWM9	8.891	8.897	8.910	8.896	2.820	8.899	4.479	0.721	0.365
NWM11	11.233	10.716	10.728	10.756	10.876	10.839	10.733	10.739	10.744

NWM11 holds its claimed order of 10.7446 across the entire eccentricity range, including the corner where it also fails most often — when it converges there, it converges at full order. Danby is likewise immovable at 4.000.

NWM9 degrades monotonically beyond $e \approx 0.99$, falling to 0.365 at $e = 0.999$: sub-linear, effectively a different algorithm. Two caveats apply. The value at $e = 0.9$ (2.820) is *not* part of that trend — its neighbours at 0.7 and 0.95 are both ≈ 8.9 — and a non-monotonic dip

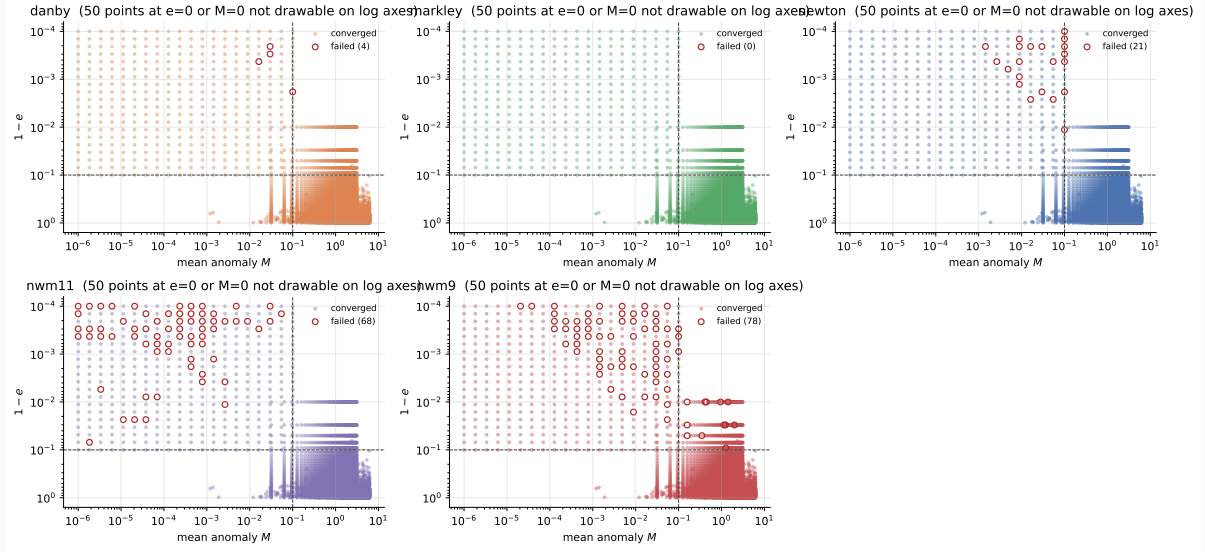


Figure 6: Where each solver fails, on logarithmic axes in $1 - e$ and M so the pathological corner occupies the top left rather than a sliver. Failures are almost entirely confined to that corner. The totals track the regional rates above: Markley 0, Danby 4, Newton 21, NWM11 68, NWM9 78. Points at exactly $e = 0$ or $M = 0$ cannot be drawn on log axes and are noted in each panel title.

that recovers is far more likely an estimator artefact at a single point than a property of the method. Newton’s missing entry at $e = 0.995$ and its 1.786 at $e = 0.999$ are the same kind of artefact; Newton’s order does not genuinely change.

7 DOWNSTREAM: DOES ANY OF IT MATTER?

Solver error was traced through $M \rightarrow E \rightarrow v \rightarrow v_r$ into refitted orbital parameters, using RadVel on the HD 164922 dataset (401 HIRES points across three instrument eras). Each fit performs roughly 1.35×10^5 Kepler solves.

7.1 COST OF A FULL FIT

Solver	Median fit (s)	Slowdown vs. native	μ s per solve
Newton	1.32	16.7×	9.89
Danby	1.43	18.0×	10.73
Markley	1.54	19.4×	11.96
NWM11	2.06	25.7×	15.12
NWM9	2.33	29.1×	16.81

The slowdown against RadVel’s native solver is dominated by the scalar-Python bridge — our solvers are scalar and RadVel passes arrays — so these figures compare our implementations against each other, not against production code. On that basis the with-memory schemes are again the slowest, consistent with Section 5.

7.2 THE ERROR BUDGET

The question is not how large the solver-induced parameter shift is, but how it compares to the two uncertainties that are already present.

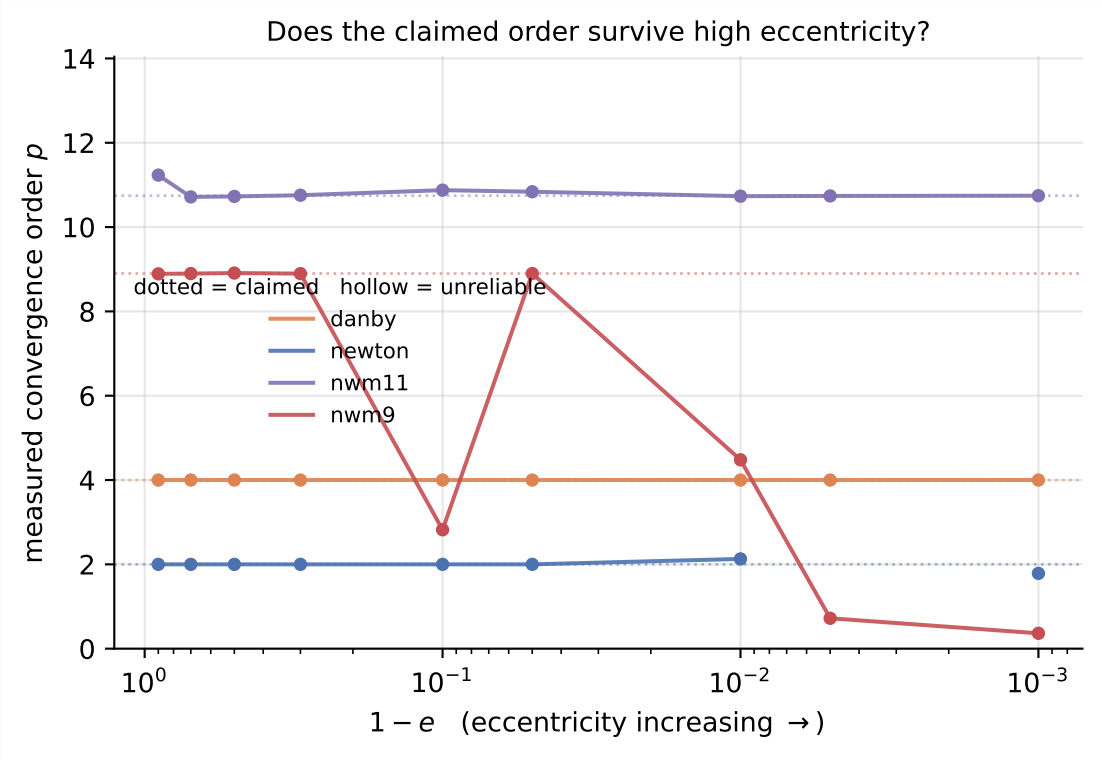


Figure 7: Measured order against eccentricity. Dotted lines mark each method’s claimed order; hollow markers indicate estimates the measurement could not resolve. NWM11 tracks its claim across the full range while NWM9 collapses beyond $e \approx 0.99$.

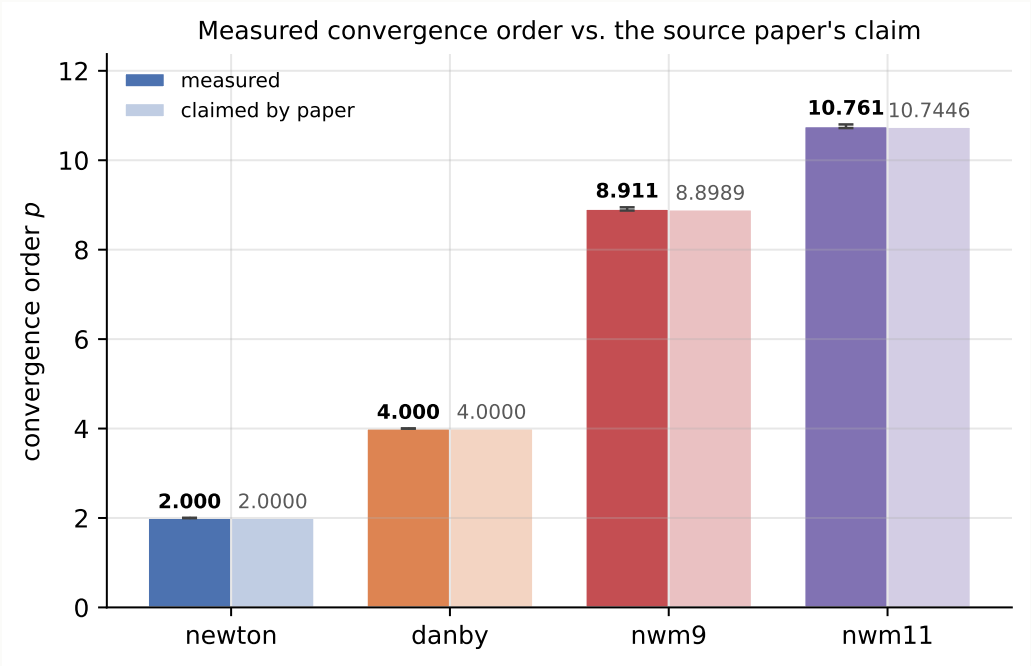


Figure 8: Measured convergence order against each source paper’s claim on the paper’s own test functions. Error bars show the spread across the eight functions.

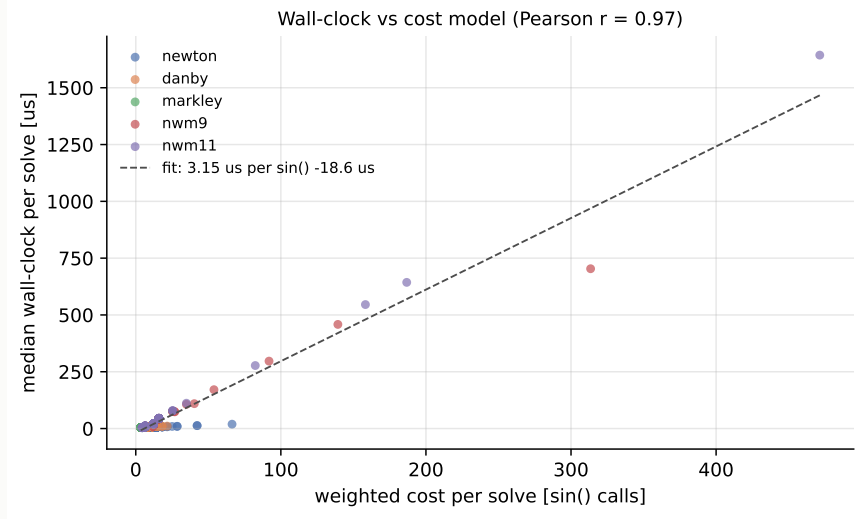


Figure 9: Median wall-clock time against the weighted cost model, one point per solver-guess-region combination. The two track each other closely across the full range — Pearson $r = 0.97$, fitted at $3.15 \mu\text{s}$ per unit cost — so the cost model is a good predictor of time *in aggregate*. That coexists with the median ranking in Section 5 inverting between the two measures, because the with-memory schemes sit at low cost and high per-iteration overhead simultaneously: a strong overall correlation does not force the ordering to agree at any particular point.

Parameter	Solver shift (tol = 10^{-4})	Noise spread (Monte Carlo)	MCMC posterior (1σ)	Noise / solver
P	4.9×10^{-9}	—	6.4×10^{-4}	—
e	1.1×10^{-5}	1.58×10^{-2}	1.73×10^{-2}	1.4×10^3
ω	2.3×10^{-5}	5.34×10^{-2}	5.39×10^{-2}	2.4×10^3
K	3.6×10^{-5}	1.06×10^{-1}	1.01×10^{-1}	3.0×10^3

SOLVER CHOICE IS NOT A SOURCE OF ERROR IN PRACTICE

Even at the loosest tolerance tested (10^{-4} , far looser than any sensible production setting), the shift a solver induces in the fitted parameters is **three orders of magnitude smaller** than the scatter caused by measurement noise in the data — and the noise-driven spread independently reproduces RadVel’s own MCMC posterior width to within 10%, which is a useful consistency check on the Monte Carlo study itself.

At the tolerances anyone would actually use (10^{-12} and below) the shift is at or below the 10^{-9} level, and at 10^{-14} it is identically zero.

The practical consequence: *choose the Kepler solver for speed and robustness, not for accuracy.* Accuracy stopped being the binding constraint long before it reached the astronomy.

8 SYNTHESIS

Returning to the question the proposal posed — under which conditions of eccentricity, mean anomaly, and Kepler-specific evaluation cost is each class of method preferable?

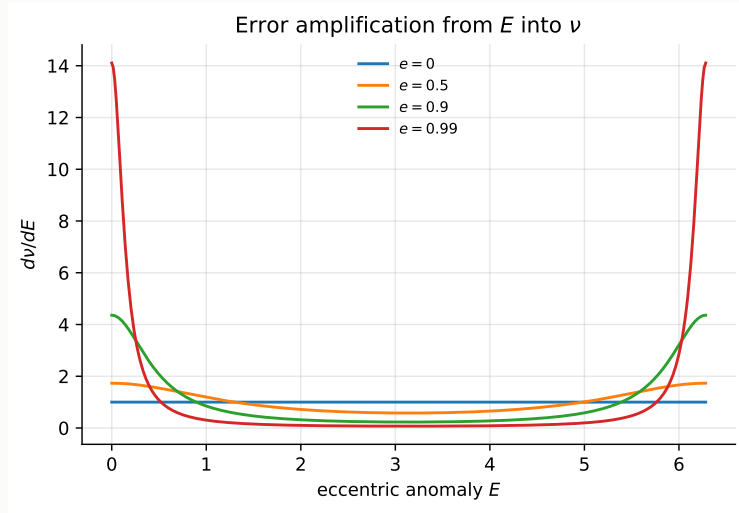


Figure 10: The amplification dv/dE carrying solver error from the eccentric anomaly into the true anomaly. Near-circular orbits pass error through roughly unchanged; at $e = 0.99$ the factor exceeds 14 at the ends of the range. This is the mechanism by which a solver error could in principle matter — and the measured budget below is what shows it does not.

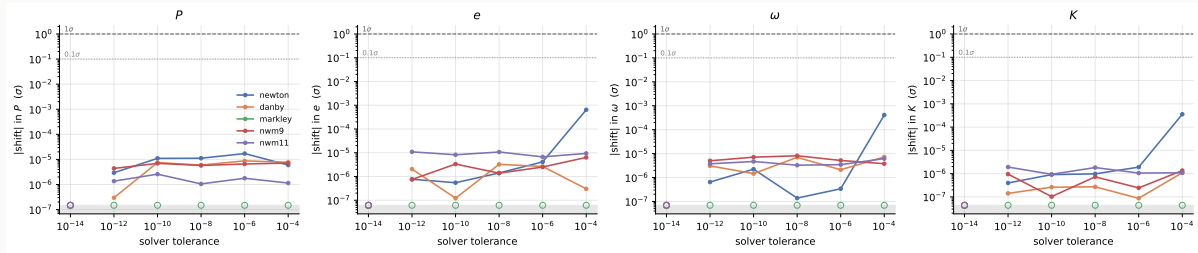


Figure 11: Shift in each fitted parameter as the solver tolerance is loosened, in units of the MCMC posterior σ . The dotted guides mark 1σ and 0.1σ . Every curve stays four to eight orders of magnitude below 1σ across the whole sweep, and collapses to exactly zero at 10^{-14} , the reference tolerance.

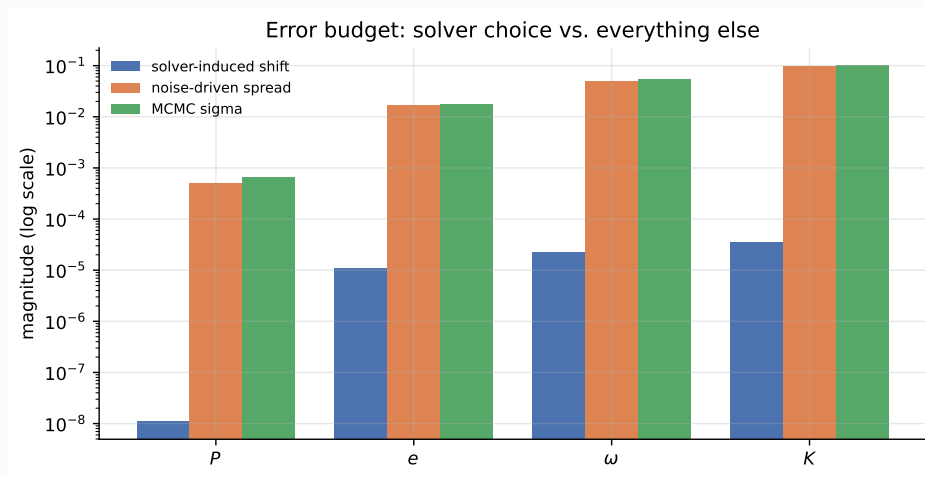


Figure 12: The three-way budget on a log scale: solver-induced shift against noise-driven spread against the MCMC posterior width. The second and third bars agree closely for every parameter, which is an independent check on the Monte Carlo study; the first is smaller by three orders of magnitude.

Axis	Verdict
Convergence order	With-memory wins decisively and as advertised. NWM11 measures 10.76 against Danby's 4.00, and holds it across the whole eccentricity range.
Iteration count	With-memory wins: a median of 1 iteration against Danby's 2 and Newton's 3–4.
Kepler-specific weighted cost	With-memory wins, 6.44 against 7.88 — about 18% cheaper than RadVel's production solver.
Wall-clock time	With-memory <i>loses</i> , by 2–5 $\times$. The interpolation is free in transcendental calls and expensive in arithmetic.
Robustness, ordinary region	Tie, or a slight with-memory win: NWM11 records zero failures in 28,076 solves.
Robustness, pathological corner	With-memory <i>loses</i> badly: 5.2% failure against Danby's 0.20%.
Downstream accuracy	Irrelevant. All methods are 10^3 times more accurate than the data justify.

The trade-off falls exactly where the proposal anticipated it might — the newer variants win on order and iteration count and lose on per-iteration cost and high-eccentricity robustness — but with one result that was not anticipated. **The Kepler-specific cost advantage, which was the project's whole reason for expecting the with-memory schemes to compete, is real in the cost model and absent from the clock.** The model assumes a `sincos` discount that this implementation does not provide.

That is not a negative result about the methods. It is a statement about where they would pay off: in a compiled kernel with a genuine fused `sincos`, the 18% weighted-cost advantage should convert into real time. Testing that claim requires a compiled implementation and is the natural continuation of this work.

Markley's closed-form solver, meanwhile, was the quiet winner throughout: cheapest on the cost model at 3.44, joint-fastest on the clock, and the only method in the study with a zero failure rate anywhere on the grid.

9 THREATS TO VALIDITY

1. **Pure-Python arithmetic.** Every wall-clock figure is measured through the CPython interpreter, where arithmetic is expensive relative to transcendental calls and no fused `sincos` exists. This is the single largest threat, and it works specifically against the with-memory schemes.
2. **The residual-evaluation artefact.** The shared loop's own $f(E)$ call inflates every solver's `sin`-only count by one per iteration. It is uniform across methods and does not affect rankings, but the absolute per-iteration figures in Section 4 are one `sin` high.
3. **Single downstream dataset.** The error-propagation and Monte Carlo studies use HD 164922 only. The three-orders-of-magnitude margin is wide enough that the conclusion is unlikely to be dataset-specific, but it has not been tested on another system.
4. **Estimator artefacts at extreme eccentricity.** Several entries in the order-versus-eccentricity table are measurement artefacts rather than method behaviour, as noted in Section 6. They are reported rather than removed, with the unreliable points marked.
5. **NWM9's degradation is single-sourced.** The collapse beyond $e \approx 0.99$ has been reproduced across precisions but not cross-checked against an independent implementation of the same published scheme.

10 CONCLUSION

Five root finders were implemented against one instrumented interface, each verified against its source paper’s convergence order and error constant before being benchmarked, then run over 125,800 solves spanning the full (e, M) range and traced downstream into refitted orbital parameters.

The 2024–2025 with-memory schemes do what their papers claim. On Kepler’s equation they reach tolerance in fewer iterations than any iterative alternative and, on the Kepler-specific cost model the proposal argued for, they are about 18% cheaper than RadVel’s production solver. That advantage does not survive translation into wall-clock time in a pure-Python implementation, because the cost model’s central assumption — a cheap fused sincos — does not hold there. They are also markedly less reliable in the high-eccentricity corner, by a structural mechanism traced to the accelerating parameter itself.

For RadVel’s actual operating range the practical recommendation is unchanged: Danby, or Markley where a closed form is acceptable. The with-memory schemes are worth revisiting in a compiled kernel, where the cost structure the proposal described genuinely exists.

The outcome was not fixed in advance, and it did not come out uniformly in either direction. That was the point.

REFERENCES

- [1] B. J. Fulton, E. A. Petigura, S. Blunt, and E. Sinukoff, “RadVel: The Radial Velocity Modeling Toolkit,” *Publications of the Astronomical Society of the Pacific*, vol. 130, no. 986, p. 044504, 2018. doi: [10.1088/1538-3873/aaaaa8](https://doi.org/10.1088/1538-3873/aaaaa8).
- [2] J. M. A. Danby, “The solution of Kepler’s equation, III,” *Celestial Mechanics*, vol. 40, nos. 3–4, pp. 303–312, 1987. doi: [10.1007/BF01235847](https://doi.org/10.1007/BF01235847).
- [3] F. L. Markley, “Kepler equation solver,” *Celestial Mechanics and Dynamical Astronomy*, vol. 63, no. 1, pp. 101–111, 1995. doi: [10.1007/BF00691917](https://doi.org/10.1007/BF00691917).
- [4] S. K. Mittal, S. Panday, and L. Jäntschi, “Enhanced Ninth-Order Memory-Based Iterative Technique for Efficiently Solving Nonlinear Equations,” *Mathematics*, vol. 12, no. 22, p. 3490, 2024. doi: [10.3390/math12223490](https://doi.org/10.3390/math12223490).
- [5] S. K. Mittal, S. Panday, L. Jäntschi, and L. C. Bolunduț, “Two novel efficient memory-based multi-point iterative methods for solving nonlinear equations,” *AIMS Mathematics*, vol. 10, no. 3, pp. 5421–5443, 2025. doi: [10.3934/math.2025250](https://doi.org/10.3934/math.2025250).
- [6] K. J. Napier, “Improved Initial Guesses for Numerical Solutions of Kepler’s Equation,” *arXiv preprint arXiv:2411.15374*, 2024. doi: [10.48550/arXiv.2411.15374](https://doi.org/10.48550/arXiv.2411.15374).
- [7] E. A. Petigura *et al.*, “Two Transiting Low Density Sub-Saturns from K2,” *The Astronomical Journal*, vol. 153, no. 4, p. 142, 2017. (*HD 164922 and K2-24 system parameters; see data/README.md for the full dataset provenance.*)